

UNIVERSIDAD EUROPEA DE VALENCIA
Grado en Física

Subject: Introduction to artificial intelligence

NewtonRapson on Semiconductor Physics

Author:
Pau Gallego Cisteró

Date:
May 11, 2026

CONTENTS

1	Summary and objectives	2
2	Principles	2
3	Questions	3
4	References	6

1. SUMMARY AND OBJECTIVES

The goal of this activity is to study the quantum confinement of electrons in a finite potential well, a fundamental concept in semiconductor physics. Using numerical methods, we solve the time-independent Schrödinger equation to determine the discrete energy levels allowed within the well.

The specific objectives are:

- Plot the energy spectrum of a finite potential well with width $a = 3 \text{ \AA}$ and height $V_0 = 3 \text{ eV}$.
- Find the first two allowed energy levels within the well using Newton–Raphson root-finding applied to the transcendental equations.
- Determine the correct effective mass $m^* = x \cdot m_e$ that reproduces the expected energy spectrum for the given semiconductor.
- Discuss the relationship between the depth of the potential well and the number and position of the allowed energy levels.

2. PRINCIPLES

A finite potential well is a quantum system in which a particle is confined within a region of lowered potential $-V_0$, but unlike the infinite well, there is a nonzero probability of finding the particle outside the well due to quantum tunneling. The system is governed by the time-independent Schrödinger equation:

$$-\frac{\hbar^2}{2m} \frac{d^2\psi(x)}{dx^2} + V(x)\psi(x) = E\psi(x), \quad (1)$$

where $V(x)$ represents the potential, $\psi(x)$ is the wave function, and E is the particle energy.

For a well of width a and height V_0 , the solutions inside and outside the well lead to transcendental equations that must be solved numerically. Introducing the dimensionless variables $\xi = ka$ and $\eta = \beta a$, and the parameter

$$r^2 = \frac{2m}{\hbar^2} V_0 a^2, \quad (2)$$

the allowed energy levels correspond to the intersection points of the following pairs of equations. For the even (symmetric) solutions:

$$\xi \tan(\xi) = \eta, \quad \xi^2 + \eta^2 = r^2. \quad (3)$$

For the odd (antisymmetric) solutions:

$$-\xi \cot(\xi) = \eta, \quad \xi^2 + \eta^2 = r^2. \quad (4)$$

The energy of each level is recovered from the root ξ via

$$E = \frac{\hbar^2 \xi^2}{2m^* a^2}. \quad (5)$$

3. QUESTIONS

Plot the energy spectrum of the finite potential well with $a = 3 \text{ \AA}$ and $V_0 = 3 \text{ eV}$, and find the first two allowed energy levels within the well.

```
1
2 import numpy as np
3 import sympy as sp
4 import matplotlib as mp
5 import matplotlib.pyplot as plt
6
7
8 V0=3
9 a=3
10 h_bar= 1.0545718e-34 # J*s
11 # r = np.sqrt( (2*m_eff*V0*a**2) / h_bar**2 )
12 z0 = 10.0
13 r = z0
14
15
16
17 x = sp.symbols('x', real=True, positive=True) # define la variable
    simbolica x
18
19 #Paridad definida para las funciones de onda:
20
21 # Funcion Par
22 def f_par_sym(x, r):
23     return x * sp.tan(x) - sp.sqrt(r**2 - x**2)
24
25 # Funcion Impar
26 # cot(x) = 1/tan(x)
27 def f_impar_sym(x, r):
28     return x / sp.tan(x) + sp.sqrt(r**2 - x**2)
29
30
31 f_par = f_par_sym(x, r)
32 f_impar = f_impar_sym(x, r)
33 f_d_par = sp.diff(f_par, x)
34 f_d_impar = sp.diff(f_impar, x)
35
36
37 f = sp.lambdify(x, f_par)
38 f_d = sp.lambdify(x, f_d_par)
39 f_imp = sp.lambdify(x, f_impar)
40 f_d_imp = sp.lambdify(x, f_d_impar)
41
42
43 # Newton-Rapson PAR
44 N_max = 20
45 tol = 1.e-9
46 raices_totales = []
47
48 # Calculamos cuantos intervalos de pi/2 hay hasta z0
49 n_intervalos = int(np.ceil(z0 / (np.pi/2)))
```

```

50
51 def seed_par(inf, sup, N=1000):
52     for _ in range(N):
53         x_random = np.random.uniform(inf + 0.05, sup - 0.05)
54         if np.abs(f(x_random)) < 5.0:
55             return x_random
56     return (inf + sup) / 2
57
58 # Recorremos solo los intervalos pares
59 for n in range(0, n_intervalos, 2):
60     limite_inf = n * np.pi / 2
61     limite_sup = min((n + 1) * np.pi / 2, z0)
62
63     if limite_inf >= limite_sup: break
64
65     x_aprox = [seed_par(limite_inf, limite_sup)]
66
67     for k in range(1, N_max):
68         df = f_d(x_aprox[k-1])
69         if np.abs(df) < 1e-14:
70             break
71         x_new = x_aprox[k-1] - f(x_aprox[k-1]) / df
72         x_aprox.append(x_new)
73         if x_new >= z0:
74             break
75
76         if np.abs(x_new - x_aprox[k-1]) / np.abs(x_new) < tol:
77             if limite_inf <= x_new <= limite_sup:
78                 raices_totales.append(x_new)
79                 print('Raiz PAR: ', x_aprox[-1])
80                 break
81
82 def seed_impar(inf, sup, N=1000):
83     for _ in range(N):
84         x_random = np.random.uniform(inf + 0.05, sup - 0.05)
85         if np.abs(f_imp(x_random)) < 5.0:
86             return x_random
87     return (inf + sup) / 2
88
89 # Recorremos solo los intervalos impares
90 for n in range(1, n_intervalos, 2):
91     limite_inf = n * np.pi / 2
92     limite_sup = min((n + 1) * np.pi / 2, z0)
93
94     if limite_inf >= limite_sup: break
95
96     x_aprox_impar = [seed_impar(limite_inf, limite_sup)]
97
98     for k in range(1, N_max):
99         df = f_d_imp(x_aprox_impar[k-1])
100         if np.abs(df) < 1e-14:
101             break
102         x_new = x_aprox_impar[k-1] - f_imp(x_aprox_impar[k-1]) / df
103         x_aprox_impar.append(x_new)
104         if x_new >= z0:
105             break
106

```

```

107         if np.abs(x_new - x_aprox_impar[k-1]) / np.abs(x_new) < tol:
108             if limite_inf <= x_new <= limite_sup:
109                 raices_totales.append(x_new)
110                 print('Raiz IMPAR: ', x_aprox_impar[-1])
111                 break
112
113 # Niveles de E
114 print('\nAPARTADO 1: Niveles de energia dentro del pozo')
115 raices_totales.sort() # Ordenar las raices
116 for i, chi in enumerate(raices_totales):
117     # E = V0 * (chi / z0)^2
118     energia_ev = V0 * (chi / z0)**2 # Chi = raices
119     print(f'Nivel {i+1}: E = {energia_ev:.4f} eV')
120
121
122 mp.__version__
123
124 xx = np.linspace(0.01, z0, 2000)
125
126 yy_tan = np.tan(xx)
127 yy_cot = -1/np.tan(xx)
128
129 yy_circ = np.zeros_like(xx)
130 mask = xx <= r
131 yy_circ[mask] = np.sqrt((r/xx[mask])**2 - 1)
132 yy_circ[~mask] = np.nan
133
134 fig = plt.figure(figsize = (10,8))
135 plt.ylim(0, 10)
136 plt.xlim(0, z0)
137
138 plt.plot(xx, yy_circ, c='r', lw=2, label='f(x) =  $\sqrt{(z_0/x)^2 - 1}$ ')
139 plt.plot(xx, yy_tan, c='b', lw=1.5, label='g(x) =  $\tan(x)$ ')
140 plt.plot(xx, yy_cot, c='g', lw=1.5, label='g(x) =  $-1/\tan(x)$ ')
141
142 for raiz in raices_totales:
143     y_raiz = np.sqrt((r/raiz)**2 - 1)
144     plt.plot(raiz, y_raiz, 'ko', markersize=8, zorder=5)
145
146 plt.xlabel('z (eV)', fontsize=12)
147 plt.ylabel('Functions', fontsize=12)
148 plt.grid(True, linestyle=':', alpha=0.6)
149 plt.legend(loc='upper right')
150
151 plt.show()

```

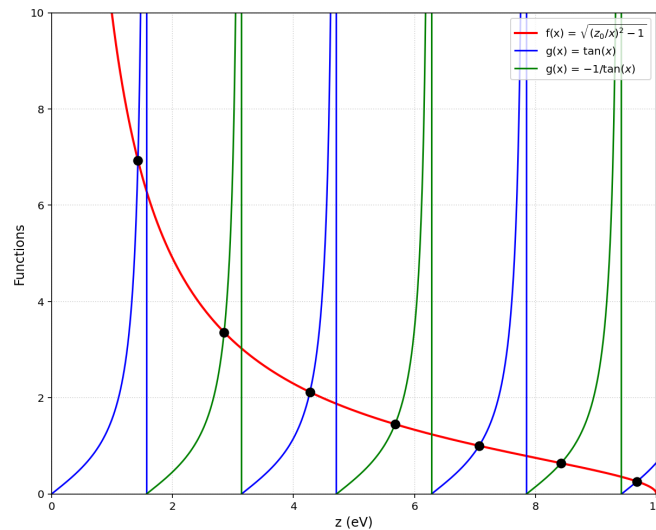


Figure 1

Find the correct effective mass $m^* = x \cdot m_e$, where m_e is the electron mass, that reproduces the expected energy levels for this semiconductor.

```

1 # Despejamos m_eff (m*):
2 print('\nAPARTADO 2: Massa efectiva')
3 m_eff = (z0**2 * h_bar**2) / (2 * V0 * a**2)
4 print('La masa efectiva es: ', m_eff)
5 print('

```

Explain the relationship between the energy levels and the depth of the potential well V_0 .

La profundidad del pozo de potencial, V_0 , actúa como el factor determinante tanto para la cantidad de estados permitidos como para el grado de confinamiento de los portadores de carga. Un incremento en V_0 aumenta el valor del parámetro adimensional $r = \sqrt{2m^*V_0a^2/\hbar^2}$, lo que expande el radio de la solución gráfica. Esto se traduce en la aparición de nuevos niveles de energía discretos dentro del pozo, indicando que el semiconductor puede albergar más estados cuantizados a medida que el potencial se vuelve más profundo.

4. REFERENCES

- [1] La Mecánica Cuántica. (2009, August 11). *Transmisión y reflexión de partículas IV: Pozo de potencial rectangular de profundidad finita*. Recuperado el 6 de Mayo, de https://la-mecanica-cuantica.blogspot.com/2010/07/transmision-y-reflexion-de-particulas_05.html

- [2] GEI-CAL. (s.f.). *Derivación numérica*. Cálculo 1 UDC. Recuperado el 8 de Abril, de <https://gei-cal.github.io/JB-Calculo1-UDC/capitulos/05/05.Derivacion.html>